`mov` is Turing-complete

Stephen Dolan
Computer Laboratory, University of Cambridge
stephen.dolan@cl.cam.ac.uk

Abstract

It is well-known that the `x86` instruction set is baroque, overcomplicated, and redundantly redundant. We show just how much fluff it has by demonstrating that it remains Turing-complete when reduced to just one instruction.

The instruction we choose is `mov`, which can do both loads and stores. We use no unusual addressing modes, self-modifying code, or runtime code generation. Using just this instruction (and a single unconditional branch at the end of the program to make nontermination possible), we demonstrate how an arbitrary Turing machine can be simulated.

registers for now, but later we show how their number can be reduced without losing expressiveness.

We have the following instructions (if you like RISC) or addressing modes (if you like CISC). We use Intel `x86` syntax, where the `mov` instructions have the destination first and the source second, and square brackets indicate memory operands.

Instruction	`x86` **syntax**
Load Immediate	`mov` $R_{\texttt{dest}}$`, c`
Load Indexed	`mov` $R_{\texttt{dest}}$`, [`$R_{\texttt{src}}$ `+` $R_{\texttt{offset}}$`]`
Store Indexed	`mov [`$R_{\texttt{dest}}$ `+` $R_{\texttt{offset}}$`],` $R_{\texttt{src}}$

On `x86`, these are all addressing modes of the same `mov` in-